

자료구조 실습 보고서

[제 01주] 마방진

제출일 : 2018년 3월 10일

201702081 최재범

1. 프로그램 설명서

(1) 주요 알고리즘

- 마방진 채우기

마방진을 채우는 과정은 크게 목적지 설정, 이동, 값 대입의 세 가지 과정으로 나눌 수 있다.

이 세 과정은 for문 속에 위치하여, 한 번씩 순회할 때마다 대입할 값이 1씩 증가하며 차수에 따라 마방진의 크기에 맞도록 대입 요소의 제한값이 정해진다.

반복문의 순회가 끝나면 완성된 마방진을 포함하는 Board 클래스의 인스턴스를 반환한다.

① 목적지 좌표 설정

➔ 먼저 임시 변수에 일반적인 이동 후의 좌표를 넣는다. 일반적으로 오른쪽 위로 이동하므로, 행 인덱스를 1 감소, 열 인덱스를 1 증가시키면 된다.

만일 현재의 좌표가 마방진의 모서리 부분이라면, 이를 반영하여 마방진 틀들을 이어붙였을 때로 가정하고 맨 왼쪽 혹은 맨 아래 인덱스 쪽으로 지정한다.

그 후, 임시변수 값을 미리 만들어놓은 위치 정보에 대입함으로써 다음 위치를 설정한다.

만일 설정된 다음 위치에 대입된 값이 이미 존재할 시, 열 번호를 1 증가시킴으로써 한 칸 아래로 이동시킨다.

② 목적지로 이동

➔ 현재 위치에 이동할 위치의 좌표를 대입한다.

③ 값 대입

➔ 현재 위치에 값을 대입한다.

(2) 자료구조

- AppController 클래스

① 클래스 사용을 위한 _appView, _magicSquare 필드가 존재한다.

② AppView 클래스를 이용하여 입출력을, MagicSquare 클래스를 이용하여 입력값 판단 및 마방진 계산을, CellLocation 클래스를 사용하여 전체 마방진 출력을 구현한다.

- Board 클래스

① 차수 저장을 위한 _order, 마방진의 구조가 될 2차원배열의 사용을 위한 _cell 필드가 존재한다.

② CellLocation 클래스를 이용하여 좌표를 구성한다.

- MagicSquare 클래스

① 디폴트 최대 차수를 결정하는 DEFAULT_MAX_ORDER, 최대 차수를 저장하는 _maxOrder 필드가 존재한다.

② Board 클래스와 CellLocation 클래스를 이용하여 완성된 마방진을 만든다.

- AppView 클래스

① Scanner 사용을 위한 `_scanner` 필드가 존재한다.

- `CellLocation` 클래스

① 행, 열을 저장하는 `_row`, `_col` 필드가 존재한다.

(3) 함수 설명서

- `AppController` : 전체적인 제어를 담당

① `public void run()`

➔ 시작 메시지를 출력하고, 마방진 크기를 입력받은 후, 입력값을 판단하여 계산 후 출력하기까지의 총체적인 명령만을 담당한다.

② `private void showOrderValidityErrorMessage(OrderValidity anOrderValidity)`

➔ 유효성을 나타내는 `OrderValidity`형의 요소 중 하나를 인자로 받아, 그 경우에 해당하는 메시지를 출력한다.

③ `private void showTitleForColumnIndexes(int anOrder)`

➔ 차수를 인자로 받아, 마방진 테두리 중 열의 인덱스를 나타내는 부분을 출력한다.

④ `private void showBoard(Board aBoard)`

➔ 마방진의 틀을 나타내는 `Board` 클래스의 인스턴스를 인자로 받아 마방진의 전체 형태를 출력한다.

- `AppView` : 입출력을 담당

① `public int inputOrder()`

➔ 메시지 출력 후 마방진의 차수(정수형)를 사용자로부터 입력받아 반환한다.

② `public void output(String outString)`

➔ 문자열을 인자로 받아 출력한다.

③ `public void outputLine(String outString)`

➔ 문자열을 인자로 받아 출력한 후 줄을 바꾼다.

④ `public void outputTitleWithOrder(int order)`

➔ 차수를 인자로 받아, 마방진의 제목을 차수와 함께 출력한다.

⑤ `public void outputRowNumber(int rowNumber)`

➔ 정수를 인자로 받아, [] 안에 넣어 출력한다. (마방진 테두리 중 행 인덱스 출력에 사용)

⑥ `public void outputCell(int cellValue)`

➔ 정수를 인자로 받아, 앞뒤에 공백을 붙여 출력한다. (마방진 요소 출력에 사용)

- `MagicSquare` : 마방진의 유효성 판단, 계산

① `public int maxOrder()`

➔ 가능한 최대 차수를 지정해놓은 `_maxOrder` 필드를 반환한다. (getter)

② `public OrderValidity checkOrderValidity(int anOrder)`

➔ 차수를 입력받아, 해당하는 유효성에 맞는 `OrderValidity`형의 요소를 반환한다.

③ public Board solve(int anOrder)

➔ 차수를 입력받아, 모든 값을 채운 Board 클래스의 인스턴스를 반환한다.

- Board : 마방진의 틀을 나타냄

① public int order()

➔ 차수를 나타내는 _order 필드를 반환한다. (getter)

② public int cell(CellLocation aLocation)

➔ CellLocation의 인스턴스 형태로 좌표를 받아, 그 요소의 값을 반환한다. (getter)

③ public int cell(int aRow, int aCol)

➔ 요소의 좌표를 행, 열 순으로 인자로 받아, 그 요소의 값을 반환한다. (getter)

④ public void setCell(CellLocation aLocation, int aNumber)

➔ CellLocation의 인스턴스 형태로 좌표를 받아, 인자로 받은 값을 그 곳에 넣는다. (setter)

⑤ public void setCell(int aRow, int aCol, int aNumber)

➔ 좌표를 행, 열 순으로 인자로 받아, 인자로 받은 값을 그 곳에 넣는다. (setter)

⑥ public boolean cellsEmpty(CellLocation aLocation)

➔ CellLocation의 인스턴스 형태로 좌표를 받아, 그 곳이 비어있는지 확인한 후 참, 거짓을 반환한다.

- CellLocation : 마방진 원소의 좌표를 나타냄

① public int row()

➔ 행을 나타내는 _row 필드를 반환한다. (getter)

② public int col()

➔ 열을 나타내는 _col 필드를 반환한다. (getter)

③ public void setRow(int newRow)

➔ 행 번호를 받아 좌표에 설정한다. (setter)

④ public void setCol(int newCol)

➔ 열 번호를 받아 좌표에 설정한다. (setter)

(4) 종합 설명서

차수를 입력받아, 그 크기에 맞는 마방진을 출력한다.

차수는 3 이상 99 이하의 홀수인 정수이어야 하며, 짝수이거나 범위에 해당되지 않는 수를 입력하면 에러를 띄우게 된다.

마방진을 출력한 후에도 계속 입력을 받으며, 종료를 하려면 음수를 입력하면 된다.

2. 프로그램 장단점 분석

(1) 장점

① 큰 차수의 마방진을 쉽게 계산할 수 있다.

② MVC구조로 설계되어, 클래스 별 역할의 구분이 비교적 명확하다.

(2) 단점

- ① nextInt()로 입력을 받기 때문에, 정수가 아닌 소수나 문자 등을 입력하면 예외가 발생한다.
- ② AppView 클래스가 아닌 곳 (AppController) 에도 출력 메소드가 존재하여 메소드 사용에 혼란이 온다.

3. 실행 결과 분석

(1) 입력과 출력

<<< 마방진 풀이를 시작합니다. >>>

마방진 차수를 입력하십시오. (음수를 입력하면 종료합니다) : 3

Magic Square Board : Order 3

	[0]	[1]	[2]
[0]	8	1	6
[1]	3	5	7
[2]	4	9	2

마방진 차수를 입력하십시오. (음수를 입력하면 종료합니다) : 1

오류 : 차수가 너무 작습니다. 3 보다 크거나 같아야 합니다.

마방진 차수를 입력하십시오. (음수를 입력하면 종료합니다) : 4

오류 : 차수가 짝수입니다. 홀수이어야 합니다.

마방진 차수를 입력하십시오. (음수를 입력하면 종료합니다) : 101

오류 : 차수가 너무 큼니다. 99 보다 작거나 같아야 합니다.

마방진 차수를 입력하십시오. (음수를 입력하면 종료합니다) : -123

<<< 마방진 풀이를 종료합니다. >>>

(2) 결과 분석

예시로 3을 입력한 결과를 보면, 가로, 세로, 대각선의 합이 항상 15로 일정하므로 잘 채워졌다는 점을 알 수 있다.

0 이상 3 미만, 100 이상의 정수를 입력하거나, 짝수를 입력하면 상황에 맞는 에러를 띄우며, 음수를 입력하면 프로그램이 정상적으로 종료된다.

4. 소스 코드

AppController.java

// 전체적 제어 정보

```
public class AppController {  
    private AppView _appView;  
    private MagicSquare _magicSquare;
```

```

// 생성자
// 기능 사용을 위한 객체 생성
public AppController() {
    this._appView = new AppView();
    this._magicSquare = new MagicSquare();
}

// 시작
public void run() {

    this._appView.outputLine("<<< 마방진 풀이를 시작합니다. >>>");
    this._appView.outputLine("");

    // 마방진 크기 입력
    int order = this._appView.inputOrder();

    while(order >= 0) {

        // 입력값 판단
        OrderValidity currentOrderValidity =
this._magicSquare.checkOrderValidity(order);

        if(currentOrderValidity == OrderValidity.Valid) {

            // 마방진 크기 표시
            this._appView.outputTitleWithOrder(order);

            // 계산 후 출력
            Board solvedBoard = this._magicSquare.solve(order);
            this.showBoard(solvedBoard);
        }

        else {
            this.showOrderValidityErrorMessage(currentOrderValidity);
        }

        // 새로운 크기 입력
        order = this._appView.inputOrder();
    }

    this._appView.outputLine("");
    this._appView.outputLine("<<< 마방진 풀이를 종료합니다. >>>");
}

// 에러 출력
private void showOrderValidityErrorMessage(OrderValidity anOrderValidity) {

    switch(anOrderValidity) {
        case TooSmall :
            this._appView.outputLine("오류 : 차수가 너무 작습니다. 3 보다
크거나 같아야 합니다.");
            break;
        case TooLarge :
            this._appView.outputLine("오류 : 차수가 너무 큼니다. 99 보다
작거나 같아야 합니다.");
            break;
    }
}

```

```

        case NotOddNumber :
            this._appView.outputLine("오류 : 차수가 짝수입니다. 홀수이어야
합니다.");
            break;
        default :
            break;
    }
}

// 마방진의 열 인덱스 출력 (showBoard()에 사용)
private void showTitleForColumnIndexes(int anOrder) {

    this._appView.output("    ");
    for(int col = 0; col < anOrder; col++) {
        this._appView.output(String.format(" [%3d]", col));
    }
    this._appView.outputLine("");
}

// 마방진 전체 출력
private void showBoard(Board aBoard) {

    CellLocation currentLoc = new CellLocation();

    // 열 인덱스 출력
    this.showTitleForColumnIndexes(aBoard.order());

    // 행 인덱스 출력
    for(int row = 0; row < aBoard.order(); row++) {
        this._appView.outputRowNumber(row);

        // 행 원소 출력
        for(int col = 0; col < aBoard.order(); col++) {
            currentLoc.setRow(row);
            currentLoc.setCol(col);
            this._appView.outputCell(aBoard.cell(currentLoc));
        }
        this._appView.outputLine("");
    }
}
}

```

AppView.java

```

// 입출력 관련

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }
}

```

```

    }

    // 마방진 차수 입력
    public int inputOrder() {
        int input;
        this.output("마방진 차수를 입력하십시오. (음수를 입력하면 종료합니다) : ");
        input = _scanner.nextInt();

        return input;
    }

    // 출력 메소드

    // 일반 출력
    public void output(String outString) {
        System.out.print(outString);
    }

    // 줄바꿈 출력
    public void outputLine(String outString) {
        System.out.println(outString);
    }

    // 마방진 크기 출력
    public void outputTitleWithOrder(int order) {
        System.out.println("Magic Square Board : Order " + order);
    }

    // [] 안에 인덱스 넣어 출력 (행 인덱스에 사용)
    public void outputRowNumber(int rowNumber) {
        System.out.printf("[%3d] ", rowNumber);
    }

    // 요소 값 출력
    public void outputCell(int cellValue) {
        System.out.printf(" %3d ", cellValue);
    }
}

```

MagicSquare.java

// 마방진의 유효성 판단, 계산에 관함

```

public class MagicSquare {

    private static final int DEFAULT_MAX_ORDER = 99;
    private int _maxOrder;

    // Getter : 가능한 최대 차수 반환
    public int maxOrder() {
        return this._maxOrder;
    }

    // 생성자 : 최대 차수 설정

```



```

public MagicSquare() {
    this._maxOrder = DEFAULT_MAX_ORDER;
}
public MagicSquare(int givenMaxOrder) {
    this._maxOrder = givenMaxOrder;
}

// 차수를 전달받아 유효성 검사 후 반환
public OrderValidity checkOrderValidity(int anOrder) {
    if(anOrder < 3) {
        return OrderValidity.TooSmall;
    }
    else if(anOrder > 99) {
        return OrderValidity.TooLarge;
    }
    else if((anOrder % 2) == 0) {
        return OrderValidity.NotOddNumber;
    }
    else {
        return OrderValidity.Valid;
    }
}

// 마방진 풀기
public Board solve(int anOrder) {

    if(this.checkOrderValidity(anOrder) != OrderValidity.Valid) {
        return null;
    }
    else {
        Board board = new Board(anOrder);

        // 처음 위치 생성 : 맨 윗줄 중앙
        CellLocation currentLoc = new CellLocation(0, anOrder / 2);

        // 다음 위치 생성
        CellLocation nextLoc = new CellLocation();

        // 출발 위치에 1 채움
        board.setCell(currentLoc, 1);

        // 다음 위치 찾아 채우기
        int lastValue = anOrder * anOrder;
        for(int cellValue = 2; cellValue <= lastValue; cellValue++) {

            // 오른쪽 위로 이동
            int destinationRow = currentLoc.row() - 1;
            int destinationCol = currentLoc.col() + 1;

            // 끝일 시
            if(currentLoc.row() == 0) {
                destinationRow = anOrder - 1;
            }
            if(currentLoc.col() == anOrder - 1) {
                destinationCol = 0;
            }

            // 다음 위치 설정
            nextLoc.setRow(destinationRow);

```

```

        nextLoc.setCol(destinationCol);

        // 다음 위치에 값이 들어있을 시 한칸 내려감
        if(!board.cellIsEmpty(nextLoc)) {
            destinationRow = currentLoc.row() + 1;
            destinationCol = currentLoc.col();
            nextLoc.setRow(destinationRow);
            nextLoc.setCol(destinationCol);
        }

        // 이동
        currentLoc.setRow(nextLoc.row());
        currentLoc.setCol(nextLoc.col());

        // 이동한 위치에 값 설정
        board.setCell(currentLoc, cellValue);
    }
    return board;
}
}
}

```

Board.java

// 마방진 틀

```

public class Board {

    private static int EMPTY_CELL = -1;

    private int _order;
    private int[][] _cell;

    // 생성자 : 마방진 틀 생성 후 초기화
    public Board(int givenOrder) {
        this._order = givenOrder;
        this._cell = new int[givenOrder][givenOrder];    // 마방진 틀 생성

        for(int row = 0; row < givenOrder; row++) {
            for(int col = 0; col < givenOrder; col++) {
                this._cell[row][col] = Board.EMPTY_CELL;
            }
        }
    }

    // Getter
    public int order() {
        return this._order;
    }

    public int cell(CellLocation aLocation) {
        return this._cell[aLocation.row()][aLocation.col()];
    }

    public int cell(int aRow, int aCol) {
        return this._cell[aRow][aCol];
    }

    // Setter
    public void setCell(CellLocation aLocation, int aNumber) {

```

```

        this._cell[aLocation.row()][aLocation.col()] = aNumber;
    }
    public void setCell(int aRow, int aCol, int aNumber) {
        this._cell[aRow][aCol] = aNumber;
    }

    // 값이 비어있는지 확인
    public boolean cellIsEmpty(CellLocation aLocation) {
        if(this._cell[aLocation.row()][aLocation.col()] == -1) {
            return true;
        }
        else {
            return false;
        }
    }
}

```

CellLocation.java

```

// 마방진 원소의 좌표

public class CellLocation {

    private int _row;
    private int _col;

    // 생성자 : 좌표 설정
    // 디폴트 : (-1, -1)
    public CellLocation() {
        this._row = -1;
        this._col = -1;
    }
    public CellLocation(int givenRow, int givenCol) {
        this._row = givenRow;
        this._col = givenCol;
    }

    // Getter
    public int row() {
        return this._row;
    }
    public int col() {
        return this._col;
    }

    // Setter
    public void setRow(int newRow) {
        this._row = newRow;
    }
    public void setCol(int newCol) {
        this._col = newCol;
    }
}

```

OrderValidity.java

// 차수에 대한 처리 결과에 관함

```
public enum OrderValidity {  
    Valid,  
    TooSmall,  
    TooLarge,  
    NotOddNumber  
}
```

DS_01_201702081_최재범.java

// main

```
public class DS_01_201702081_최재범 {  
    public static void main(String[] args) {  
        ApplicationController _appController = new ApplicationController();  
        _appController.run();  
    }  
}
```